

UNIVERSITE POLYTECHNIQUE DE GITEGA

RAPPORT DE PROJET D'EXAMEN

Développement JavaScript & Base de données NoSQL

APPLICATION DE SUIVI DE FITNESS

Groupe 3

Présenté par :

INEZA Bénisse
GAHIMBARE Benitha
EZAKO Juste Ariel
IKORIVYZA Seth

Département : Génie Logiciel BAC4

Encadré par :

MA. HAVYARIMANA Juvénal

Date de remise : 13/07/2026

Résumé

Ce projet, réalisé dans le cadre des cours de Développement JavaScript et de Bases de données NoSQL, consiste en la conception et le développement d'une application web de suivi de fitness. Construite selon une architecture 3-tiers, l'application repose sur Node.js et Express côté serveur, sur MongoDB et Mongoose pour la persistance des données, et sur JavaScript natif (vanilla) côté client.

Le modèle de données s'appuie sur sept collections (*utilisateurs*, *categories*, *exercices*, *seances*, *programmes*, *objectifs*, *historique*) reliées par des relations de référence et d'embarquement choisies selon la nature des données. La sécurité est assurée par une authentification JWT combinée au hachage bcrypt des mots de passe, avec une gestion fine des autorisations selon deux rôles : utilisateur et administrateur.

Outre les fonctionnalités obligatoires (authentification, catalogue d'exercices, séances, filtres, statistiques), nous avons implémenté plusieurs améliorations : programmes d'entraînement, objectifs personnels avec suivi de progression, et duplication de séances. L'application répond à l'ensemble des exigences fonctionnelles et techniques du cahier des charges et constitue une base solide, extensible vers de nouvelles fonctionnalités.

Table des matières

Résumé	1
1 Introduction	3
2 Analyse des besoins	3
2.1 Objectifs fonctionnels	3
2.2 Objectifs non fonctionnels	4
2.3 Cibles utilisateurs	5
3 Conception technique	5
3.1 Architecture logicielle	6
3.2 Modélisation NoSQL	6
3.3 Authentification	11
4 Implémentation	13
4.1 Stack technologique	13
4.2 Organisation du code	16
4.3 Implémentation de fonctionnalités clés	16
4.4 Tests et validation	18
4.5 Plan de tests	18
4.6 Environnement de test	19
4.7 Résultats	19
4.8 Difficultés rencontrées et solutions	21
5 Améliorations possibles	21
6 Conclusion	22
Annexes	23
Annexe A — Captures d'écran	23
Annexe B — Guide d'installation et d'exécution	23
Annexe D — Répartition des tâches	26

1 Introduction

Les cours de Développement JavaScript et de Bases de données NoSQL visent à donner aux étudiants une maîtrise complète du développement web moderne : programmation JavaScript côté client et côté serveur, conception d'API REST, gestion asynchrone des données, et modélisation de bases de données non relationnelles. Ce projet d'examen constitue l'aboutissement pratique de ces deux enseignements, en demandant la réalisation d'une application complète, du modèle de données jusqu'à l'interface utilisateur.

La problématique posée est la suivante : comment concevoir et développer une application web full-stack reposant sur une base de données NoSQL, intégrant des relations de données complexes (embarquement et référence), un système d'authentification sécurisé, et une séparation claire des responsabilités entre les différentes couches logicielles ?

Pour y répondre, nous avons choisi de développer une application de suivi de fitness, qui permet à un utilisateur de consulter un catalogue d'exercices, de composer ses propres séances d'entraînement, de les regrouper en programmes, de définir des objectifs personnels, et de suivre sa progression dans le temps au moyen de statistiques. Un rôle administrateur distinct permet de gérer le catalogue d'exercices et les comptes utilisateurs. Cette application a été développée avec Node.js, Express et MongoDB côté serveur, et en JavaScript natif côté client, conformément aux exigences du cahier des charges.

Le présent rapport est structuré comme suit : la section 2 présente l'analyse des besoins fonctionnels et non fonctionnels ; la section 3 détaille la conception technique, notamment l'architecture logicielle, le modèle de données NoSQL et le mécanisme d'authentification ; la section 4 décrit l'implémentation, les tests effectués et les difficultés rencontrées ; la section 5 propose des améliorations possibles ; enfin, la section 6 conclut sur le bilan du travail réalisé. Les annexes regroupent les captures d'écran de l'application, le guide d'installation, et la répartition des tâches au sein du groupe.

2 Analyse des besoins

2.1 Objectifs fonctionnels

TABLE 1 – Fonctionnalités obligatoires du cahier des charges

N°	Fonctionnalité	Détail
1	Authentification	Inscription, connexion, déconnexion, gestion de deux rôles (utilisateur, administrateur)

N°	Fonctionnalité	Détail
2	Catalogue d'exercices	Consultation et recherche (par nom, catégorie, muscle ciblé)
3	Séances personnalisées	Création de séances composées d'exercices choisis, avec séries / répétitions / poids / durée
4	Historique des entraînements	Conservation des séances réalisées, consultables par date
5	Statistiques	Agrégats hebdomadaires / mensuels (nombre de séances, volume soulevé, évolution)

Améliorations choisies

Elles enrichissent le modèle de données et justifient les relations complexes demandées.

TABLE 2 – Améliorations fonctionnelles implémentées

N°	Amélioration	Justification
6	Programmes d'entraînement	Regroupement de plusieurs séances sur plusieurs semaines (planification long terme)
7	Objectifs personnels	Définition d'un objectif (poids cible, performance cible) avec suivi de progression
8	Statistiques graphiques	Visualisation (Chart.js) plutôt que de simples chiffres
9	Filtres combinés	Sur le catalogue (catégorie + muscle + équipement) et sur l'historique (par date / programme)
10	Duplication de séance	Création rapide d'une nouvelle séance à partir d'une séance existante

Trois améliorations supplémentaires, non implémentées, sont par ailleurs proposées en section 5 : une version applicative mobile (PWA), un déploiement en environnement cloud, et la mise en place de tests automatisés.

2.2 Objectifs non fonctionnels

TABLE 3 – Objectifs non fonctionnels

Catégorie	Exigence
Sécurité	Mots de passe hachés (bcrypt), authentification par JWT avec expiration, middleware d'autorisation par rôle, validation des entrées côté serveur
Performance	Index MongoDB sur les champs de recherche / filtre, agrégations optimisées pour les statistiques, pagination des listes longues
Ergonomie	Interface simple, responsive, retours visuels clairs (messages d'erreur, confirmations)
Compatibilité	Navigateurs modernes (Chrome, Firefox, Edge)
Maintenabilité	Architecture en couches (modèle-vue-contrôleur), code commenté, séparation frontend / backend
Fiabilité	Gestion centralisée des erreurs, codes HTTP cohérents, validation Mongoose

2.3 Cibles utilisateurs

TABLE 4 – Rôles et droits des utilisateurs

Rôle	Description	Droits
Utilisateur (sportif)	Personne qui s'entraîne et veut suivre sa progression	Gère son profil, ses séances, ses objectifs ; consulte son historique et ses statistiques personnelles
Administrateur	Gestionnaire du contenu de référence	CRUD complet sur le catalogue d'exercices et les catégories ; gestion des comptes ; accès aux statistiques globales

3 Conception technique

3.1 Architecture logicielle

Notre application repose sur une architecture 3-tiers. Le client, constitué de pages HTML/CSS et de scripts JavaScript vanilla, communique exclusivement par requêtes HTTP (`fetch`) avec le serveur applicatif.

Le serveur, développé avec Node.js et Express, est organisé en trois couches : les routes définissent les points d'entrée de l'API REST, le middleware d'authentification valide le jeton JWT et les autorisations avant tout traitement, et les contrôleurs exécutent la logique métier en s'appuyant sur les modèles Mongoose. Cette séparation garantit qu'aucune requête non authentifiée n'atteint la base de données. Le serveur est le seul composant à dialoguer avec MongoDB, qui héberge les sept collections de l'application.

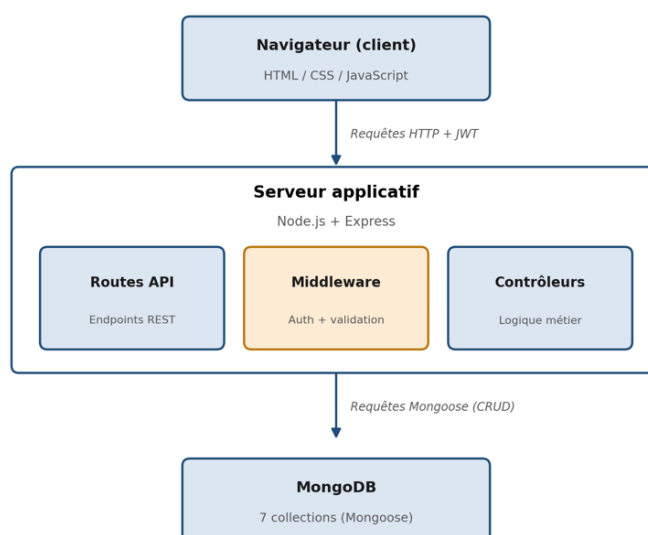


FIGURE 1 – Architecture 3-tiers de l'application

Comme l'illustre la figure 1, chaque couche a une responsabilité unique, ce qui facilite la maintenance et les tests.

3.2 Modélisation NoSQL

Le modèle de données repose sur sept collections MongoDB, choisies pour couvrir l'ensemble des fonctionnalités demandées tout en illustrant les deux types de relations possibles dans une base NoSQL : la référence et l'embarquement. La figure 2 présente une vue d'ensemble de ces collections et de leurs relations.

Détail des collections

1. utilisateurs

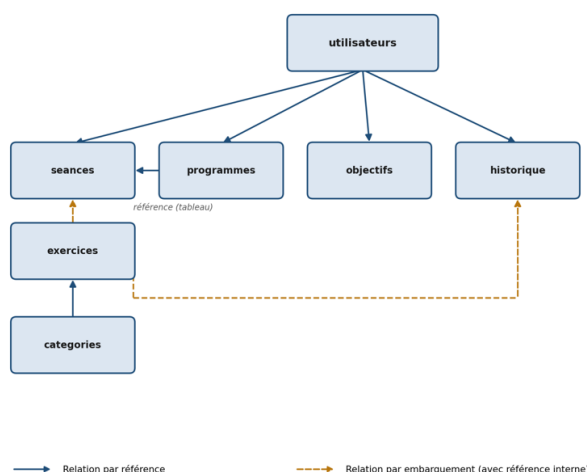


FIGURE 2 – Modèle de données — 7 collections et leurs relations

Champ	Type	Description
_id	ObjectId	Identifiant unique
nom, prenom	String	Identité
email	String, unique	Identifiant de connexion
motDePasseHash	String	Hash bcrypt, jamais le mot de passe en clair
role	String	"utilisateur" ou "admin"
profil.age / poids / taille	Number	Sous-document embarqué (toujours lu avec le compte)
actif	Boolean	Permet la désactivation par l'admin
dateInscription	Date	Horodatage

2. categories

Champ	Type	Description
_id	ObjectId	Identifiant
nom	String	Ex. "Cardio", "Haut du corps"
description	String	Détail de la catégorie

3. exercices

Champ	Type	Description
_id	ObjectId	Identifiant
nom, description, instructions	String	Contenu de l'exercice
categorie_id	ObjectId (référence)	Catégorie associée
muscleCible, équipement	String	Filtres de recherche
niveauDifficulte	String	"débutant", "intermédiaire", "avancé"
imageUrl	String	Illustration
actif	Boolean	Suppression logique

4. seances

Champ	Type	Description
_id	ObjectId	Identifiant
utilisateur_id	ObjectId (référence)	Propriétaire
nom, date	String, Date	Métadonnées
statut	String	"planifiee" / "realisee"
exercices	Array (embarqué)	{ exercice_id (réf.), series, repetitions, poids, duree }

5. programmes

Champ	Type	Description
_id	ObjectId	Identifiant
utilisateur_id	ObjectId (référence)	Propriétaire
nom, description	String	Métadonnées
dateDebut, date-Fin	Date	Durée du programme

Champ	Type	Description
seances_ids	Array d'ObjectId (référence)	Séances incluses

6. objectifs

Champ	Type	Description
_id	ObjectId	Identifiant
utilisateur_id	ObjectId (référence)	Propriétaire
type, description, unite	String	Nature de l'objectif
valeurCible, valeurActuelle	Number	Suivi
dateEcheance, statut	Date, String	"en cours" / "atteint"
progression	Array (embarqué)	{ date, valeur }

7. historique

Champ	Type	Description
_id	ObjectId	Identifiant
utilisateur_id	ObjectId (référence)	Propriétaire
seance_id	ObjectId (référence, nullable)	Séance d'origine
dateRealisation, dureeTotale	Date, Number	Métadonnées

Champ	Type	Description
exercicesRealises	Array (embarqué)	{ exercice_id (réf.), seriesRealisees, poidsUtilise, dureeReelle }

Justification des choix embarquement / référence

TABLE 12 – Justification des choix embarquement / référence

Relation	Choix	Justification
utilisateurs → seances / programmes / objectifs / historique	Référence	Collections volumineuses et interrogées indépendamment du profil ; embarquer risquerait de dépasser la limite de 16 Mo par document
categories → exercices	Référence	Une catégorie est partagée par de nombreux exercices ; la référencer évite la duplication
seances → exercices internes	Embarquement (+ réf. interne)	Séries / répétitions / poids propres à la séance, toujours lus avec elle ; seul l'exercice_id est référencé
programmes → seances	Référence	Une séance s'interroge indépendamment (historique, statistiques) ; l'embarquer dupliquerait un document volumineux
objectifs → progression	Embarquement	Petite liste de points, toujours consultée avec l'objectif
historique → exercices réalisés	Embarquement (+ réf. interne)	Snapshot figé des performances réelles, indépendant d'une modification ultérieure de la séance planifiée

Ce modèle hybride — combinant embarquement et référence au sein d'un même sous-document — illustre une utilisation réfléchie des capacités de MongoDB, plutôt qu'un choix binaire systématique.

3.3 Authentification

Nous avons retenu une authentification par JSON Web Token (JWT) plutôt que par session, pour trois raisons : un fonctionnement sans état (*stateless*), adapté à une API REST ; une intégration simple avec un frontend vanilla JS (le token se transporte dans l'en-tête **Authorization** de chaque requête `fetch`) ; et une cohérence avec MongoDB, qui n'a pas besoin d'une collection de sessions supplémentaire. Le mot de passe n'est jamais stocké en clair : il est haché avec `bcrypt` avant l'enregistrement dans la collection *utilisateurs*.

Lors de l'inscription, l'utilisateur soumet son email et son mot de passe ; ce dernier est immédiatement haché avec `bcrypt` (10 rounds de salage) avant d'être stocké. Lors de la connexion, le serveur compare le mot de passe soumis au hash stocké via `bcrypt.compare()`. Si la correspondance est valide, le serveur génère un jeton JWT avec `jwt.sign()`, signé par une clé secrète côté serveur, contenant l'identifiant de l'utilisateur et son rôle, avec une durée d'expiration de 24 heures. Ce token est renvoyé au client, qui le conserve (`localStorage`) et le joint à chaque requête protégée dans l'en-tête **Authorization: Bearer <token>**. Un middleware Express intercepte ces requêtes, vérifie la validité du token avec `jwt.verify()` et, si valide, transmet l'identité et le rôle de l'utilisateur au contrôleur. En cas de token absent, invalide ou expiré, le serveur renvoie une erreur 401 sans exécuter la logique métier.

Rappel du schéma *utilisateurs* (pertinent pour cette section)

Champ	Type	Rôle dans l'authentification
email	String, unique	Identifiant de connexion
motDePasseHash	String	Hash <code>bcrypt</code> , jamais le mot de passe brut
role	String	Encodé dans le payload du JWT, lu par le middleware d'autorisation

Tableau des routes protégées et autorisations par rôle

TABLE 14 – Routes protégées de l'API et autorisations associées

Route	Méthode	Accès	Description
/api/auth/register	POST	Public	Inscription

Route	Méthode	Accès	Description
/api/auth/login	POST	Public	Connexion, retourne le JWT
/api/users/me	GET / PUT	Utilisateur, Admin	Consulter / modifier son profil
/api/users	GET	Admin	Liste de tous les comptes
/api/users/:id/role	PATCH	Admin	Promotion / rétrogradation d'un rôle
/api/users/:id/status	PATCH	Admin	Activer / désactiver un compte
/api/categories	GET	Utilisateur, Admin	Lecture du catalogue de catégories
/api/categories	POST / PUT / DELETE	Admin	Gestion des catégories
/api/exercices	GET	Utilisateur, Admin	Lecture du catalogue d'exercices
/api/exercices	POST / PUT / DELETE	Admin	Gestion du catalogue
/api/seances	GET / POST / PUT / DELETE	Utilisateur (propriétaire)	CRUD sur ses propres séances
/api/programmes	GET / POST / PUT / DELETE	Utilisateur (propriétaire)	CRUD sur ses propres programmes
/api/objectifs	GET / POST / PUT / DELETE	Utilisateur (propriétaire)	CRUD sur ses propres objectifs

Route	Méthode	Accès	Description
/api/historique	GET / POST	Utilisateur (propriétaire)	Consultation et ajout de l'historique
/api/stats/me	GET	Utilisateur	Statistiques personnelles
/api/stats/global	GET	Admin	Statistiques globales de l'application

Remarque méthodologique : sur les routes marquées « Utilisateur (propriétaire) », le middleware effectue deux vérifications successives — la validité du token (authentification), puis la correspondance entre l'`utilisateur_id` du document demandé et celui du token (autorisation). C'est ce second contrôle qui garantit qu'un utilisateur ne peut jamais accéder aux données d'un autre.

4 Implémentation

4.1 Stack technologique

Backend

Outil / Librairie	Rôle
Node.js	Environnement d'exécution JavaScript côté serveur
Express.js	Framework web (routes, middleware)
MongoDB	Base de données NoSQL
Mongoose	ODM : schémas, validation, requêtes
bcrypt	Hachage des mots de passe
jsonwebtoken (JWT)	Génération et vérification des tokens d'authentification
dotenv	Variables d'environnement (<code>.env</code>)
cors	Autorisation des requêtes cross-origin
express-validator	Validation des données entrantes
nodemon (dev)	Redémarrage automatique du serveur en développement

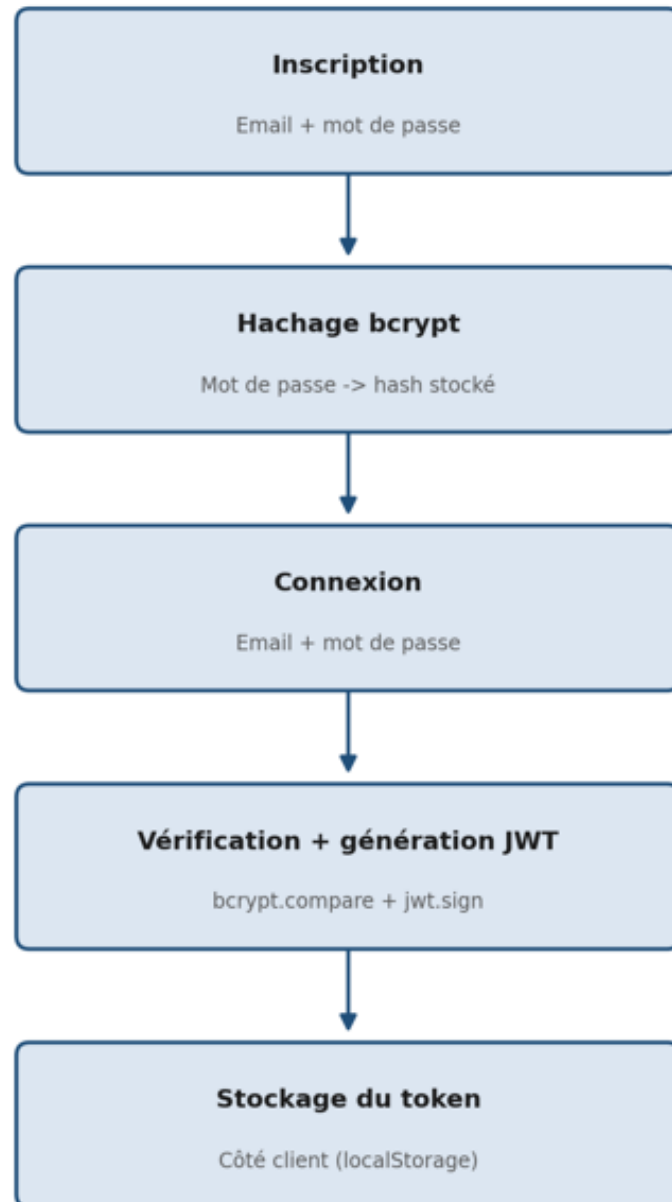


FIGURE 3 – Flux d'inscription et de connexion avec hachage bcrypt et génération du JWT

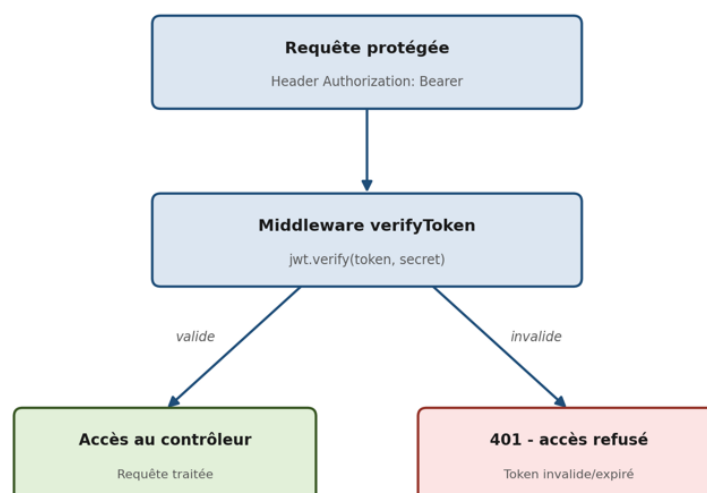


FIGURE 4 – Vérification du token JWT par le middleware d'authentification

Frontend

Outil	Rôle
HTML5 / CSS3	Structure et mise en forme des pages
JavaScript ES6+ vanilla	Logique d'interface, sans framework
Fetch API	Appels asynchrones vers l'API REST (async/await)
Chart.js	Visualisation graphique des statistiques

Outils de développement et de test

Outil	Rôle
Git / GitHub	Versionnement et dépôt à rendre
Postman	Test manuel des routes API
MongoDB Compass	Inspection visuelle des collections (optionnel)

Notre stack repose entièrement sur l'écosystème JavaScript (Node.js / Express / MongoDB), conformément aux exigences du cours, complétée par bcrypt et JWT pour la sécurité, et par Chart.js pour la visualisation des statistiques côté client.

4.2 Organisation du code

Le backend suit le principe de séparation des responsabilités : les **models** définissent la structure des données, les **routes** exposent les points d'entrée HTTP, les **controllers** contiennent la logique métier, et les **middleware** interceptent les requêtes pour l'authentification et la validation. Côté frontend, chaque ressource possède sa propre page HTML et son propre fichier JavaScript, avec un module **api.js** central qui évite de dupliquer la logique d'appel réseau et de gestion du token JWT.

```
1 fitness-app/  
2 |-- backend/  
3 | |-- config/db.js  
4 | |-- models/ (Utilisateur, Categorie, Exercice, Seance,  
5 | | Programme, Objectif, Historique)  
6 | |-- controllers/ (un fichier par ressource)  
7 | |-- routes/ (un fichier par ressource)  
8 | |-- middleware/ (authMiddleware, roleMiddleware, errorMiddleware)  
9 | |-- .env.example  
10 | |-- server.js  
11 | '-- package.json  
12 |-- frontend/  
13 | |-- index.html  
14 | |-- pages/ (login, register, dashboard, exercices, seances,  
15 | | programmes, objectifs, historique)  
16 | |-- css/style.css  
17 | '-- js/ (api, auth, exercices, seances, programmes,  
18 | | objectifs, historique, stats)  
19 '-- README.md
```

Listing 1 – Arborescence du projet

4.3 Implémentation de fonctionnalités clés

Nous présentons ci-dessous trois extraits représentatifs : une route API protégée par middleware, une agrégation MongoDB, et un appel **fetch** côté frontend.

a) Middleware de vérification du token JWT (backend/middleware/authMiddleware.js)

```
1 const jwt = require('jsonwebtoken');  
2  
3 function verifyToken(req, res, next) {  
4   const authHeader = req.headers['authorization'];  
5   if (!authHeader || !authHeader.startsWith('Bearer ')) {
```

```
6     return res.status(401).json({ success: false, error: 'Token
    manquant.' });
7 }
8 const token = authHeader.split(' ')[1];
9 try {
10     const decoded = jwt.verify(token, process.env.JWT_SECRET);
11     req.utilisateur = { id: decoded.id, role: decoded.role };
12     next();
13 } catch (err) {
14     return res.status(401).json({
15         success: false, error: 'Token invalide ou expire.',
16     });
17 }
18 }
19
20 module.exports = { verifyToken };
```

Listing 2 – Middleware de vérification du token JWT

Ce middleware est placé avant chaque contrôleur sur toutes les routes protégées. Il évite de dupliquer la logique de vérification dans chaque contrôleur, conformément au principe DRY (*Don't Repeat Yourself*).

b) Agrégation MongoDB — exercices les plus utilisés (backend/controllers/statsController)

```
1 const topExercices = await Seance.aggregate([
2     { $unwind: '$exercices' },
3     { $group: { _id: '$exercices.exercice_id', total: { $sum: 1 } } },
4     { $sort: { total: -1 } },
5     { $limit: 5 },
6     { $lookup: {
7         from: 'exercices', localField: '_id',
8         foreignField: '_id', as: 'exercice' } },
9     { $unwind: '$exercice' },
10    { $project: { nom: '$exercice.nom', total: 1 } },
11]);
```

Listing 3 – Agrégation MongoDB des exercices les plus utilisés

Le pipeline décompose le tableau embarqué **exercices** de chaque séance (**\$unwind**), compte les occurrences de chaque exercice (**\$group**), puis joint les informations du catalogue (**\$lookup**) pour obtenir le nom de l'exercice à afficher. Cette agrégation alimente le tableau de bord administrateur (route GET `/api/stats/global`).

c) Appel fetch côté frontend — création d'une séance (frontend/js/seances.js)

```

1 async function creerSeance(event) {
2   event.preventDefault();
3   const payload = {
4     nom: document.getElementById('seance-nom').value.trim(),
5     date: document.getElementById('seance-date').value,
6     exercices: exercicesSelectionnees,
7   };
8   await fetchAPI('/seances', {
9     method: 'POST',
10    body: JSON.stringify(payload),
11  });
12  await chargerSeances();
13 }

```

Listing 4 – Appel fetch pour la création d'une séance

La fonction `fetchAPI` (définie dans `api.js`) centralise l'ajout automatique du jeton JWT dans l'en-tête `Authorization` et la gestion des erreurs 401, évitant de répéter cette logique dans chaque fichier JavaScript du frontend.

4.4 Tests et validation

Les tests réalisés couvrent quatre axes : l'authentification et les autorisations par rôle, les opérations CRUD sur les sept collections, les filtres de recherche, et les fonctionnalités d'amélioration. Le plan de tests détaillé est présenté en section 4.5, l'environnement de test en section 4.6, et les résultats obtenus en section 4.7.

4.5 Plan de tests

TABLE 18 – Plan de tests

N°	Catégorie	Cas de test	Données / étapes	Résultat attendu
1	Auth	Inscription d'un nouvel utilisateur	POST /api/auth/register, email valide + mot de passe	Compte créé, mot de passe haché en base
2	Auth	Inscription avec email déjà existant	Réutiliser un email déjà inscrit	Erreur 400, « email déjà utilisé »

N°	Catégorie	Cas de test	Données / étapes	Résultat attendu
3	Auth	Connexion avec identifiants valides	POST /api/auth/login (marie.dupont@example.com)	200, retourne un JWT valide
4	Auth	Connexion avec mot de passe incorrect	Même email, mauvais mot de passe	Erreur 401, aucun token retourné
5	Auth	Connexion sur un compte désactivé	lucas.petit@example.com (actif:false)	Erreur 403, « compte désactivé »
6	Autorisation	Accès à une route protégée sans token	GET /api/seances sans en-tête Authorization	Erreur 401
7	Autorisation	Utilisateur simple sur route admin	POST /api/exercices avec le token de Marie	Erreur 403
8	Autorisation	Modifier la séance d'un autre utilisateur	Jean modifie une séance de Marie	Erreur 403/404, aucune modification
9	Exercices	Création d'un exercice par l'admin	POST /api/exercices, token admin	201, exercice visible dans le catalogue
10	Exercices	Suppression logique d'un exercice	DELETE /api/exercices/:id par l'admin	actif passe à false, disparaît du catalogue

4.6 Environnement de test

Les tests des routes API ont été réalisés avec Postman, en testant systématiquement les cas valides et invalides (token absent, rôle incorrect, données malformées). Les tests d'interface ont été effectués sur les navigateurs Google Chrome et Mozilla Firefox, en résolution desktop. La base de données de test a été initialisée à l'aide du script `seedData.js` (voir Annexe B), garantissant un jeu de données reproductible pour chaque session de test.

4.7 Résultats

TABLE 19 – Résultats des tests

N°	Cas de test	Résultat obtenu (OK/-NOK)	Correctif éventuel
1	Inscription d'un nouvel utilisateur	OK	Non Applicable
2	Inscription avec email déjà existant	NOK	Ajout d'une vérification pour empêcher les doublons d'adresse email
3	Connexion avec identifiants valides	OK	Non Applicable
4	Connexion avec mot de passe incorrect	NOK	Correction du contrôle du mot de passe et ajout d'un message d'erreur
5	Connexion sur un compte désactivé	NOK	Ajout d'une vérification du statut du compte avant l'authentification afin de bloquer l'accès aux comptes désactivés
6	Accès à une route protégée sans token	NOK	Ajout d'un middleware d'authentification JWT
7	Modifier la séance d'un autre utilisateur	NOK	Ajout d'une vérification des droits d'accès afin de permettre uniquement au propriétaire de la séance

N°	Cas de test	Résultat obtenu (OK/-NOK)	Correctif éventuel
8	Création d'un exercice par l'admin	OK	Non Applicable

4.8 Difficultés rencontrées et solutions

Difficulté 1 — Gestion des relations embarquement / référence *Cause* : déterminer pour chaque relation s'il fallait embarquer ou référencer n'était pas immédiat, notamment pour le lien entre séances et exercices.

Solution : nous avons appliqué le critère suivant : si les données sont systématiquement lues avec leur parent et propres à ce contexte (séries/répétitions d'une séance), elles sont embarquées ; si elles sont volumineuses, partagées, ou interrogées indépendamment (catalogue d'exercices), elles sont référencées.

Difficulté 2 — Isolation des données entre utilisateurs *Cause* : il fallait garantir qu'un utilisateur ne puisse jamais accéder aux séances, objectifs ou historique d'un autre utilisateur, même en connaissant l'identifiant du document.

Solution : chaque contrôleur vérifie systématiquement que le champ `utilisateur_id` du document correspond à l'identifiant contenu dans le token JWT avant toute lecture, modification ou suppression (voir le tableau des routes protégées, section 3).

Difficulté 3 — Calcul des statistiques par agrégation *Cause* : la syntaxe du pipeline d'agrégation MongoDB (`$unwind`, `$group`, `$lookup`) demande une certaine pratique, en particulier pour joindre les données du tableau embarqué `exercices` au catalogue.

Solution : nous avons décomposé le pipeline étape par étape en testant chaque étage isolément dans MongoDB Compass avant de l'intégrer au contrôleur `statsController.js`.

5 Améliorations possibles

Plusieurs pistes d'amélioration n'ont pas été implémentées dans le cadre de ce projet, par manque de temps ou parce qu'elles dépassaient le périmètre demandé. Elles sont présentées ci-dessous comme perspectives d'évolution.

- **Application mobile (PWA)** : transformer le frontend en Progressive Web App pour permettre une installation sur smartphone et un fonctionnement hors-ligne partiel.
- **Recommandations personnalisées** : suggérer des exercices ou des séances à un utilisateur en fonction de son historique et de ses objectifs.

- **Export PDF de l'historique** : permettre à un utilisateur de télécharger un bilan de ses entraînements sur une période donnée.

6 Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble des compétences travaillées durant les cours de Développement JavaScript et de Bases de données NoSQL. L'application de suivi de fitness développée répond aux exigences fonctionnelles du cahier des charges : authentification sécurisée par JWT, gestion de deux rôles distincts, sept collections MongoDB reliées par des relations de référence et d'embarquement justifiées, et un ensemble de fonctionnalités couvrant la création de séances, la planification de programmes, le suivi d'objectifs et la consultation de statistiques.

Sur le plan des compétences, ce projet a consolidé notre maîtrise de JavaScript côté serveur (Node.js, Express) et côté client (manipulation du DOM, appels asynchrones avec `fetch`), ainsi que notre compréhension de la modélisation de données non relationnelles — en particulier le choix entre embarquement et référence selon la nature et l'usage des données. La mise en place de l'authentification JWT et du hachage bcrypt nous a par ailleurs sensibilisés aux bonnes pratiques de sécurité applicative.

Le travail en groupe a nécessité une répartition claire des tâches (voir Annexe D) et une coordination régulière, notamment pour garantir la cohérence entre le modèle de données conçu collectivement et son implémentation par chaque membre. Cette organisation constitue, au même titre que le code produit, un apprentissage du projet.

Annexes

Annexe A — Captures d'écran

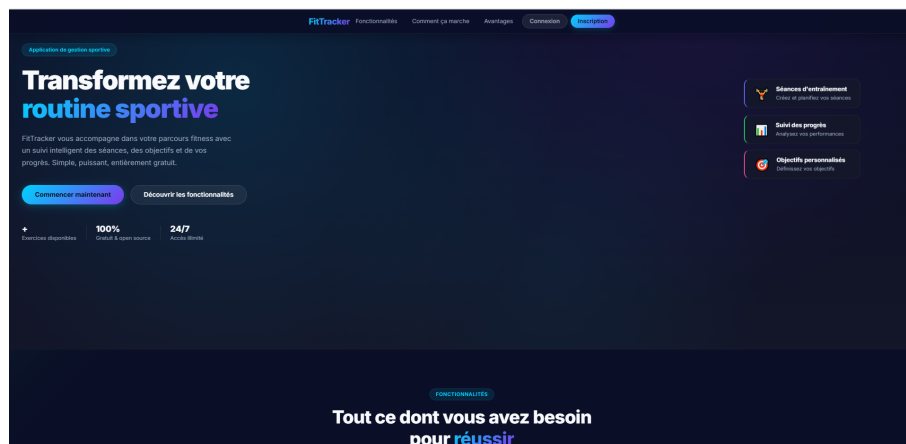


FIGURE 5 – Page d'accueil

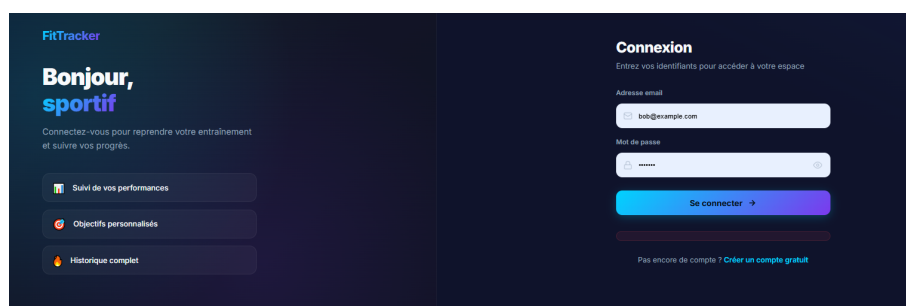


FIGURE 6 – Page de connexion

Annexe B — Guide d'installation et d'exécution

Prérequis Node.js (version 18 ou supérieure), npm, et MongoDB (local ou MongoDB Atlas).

1. Installation du backend

```
1 cd backend
2 npm install
```

2. Configuration des variables d'environnement Copier le fichier `.env.example` en `.env`, puis renseigner les valeurs :

```
1 PORT=5000
2 MONGO_URI=mongodb://localhost:27017/fitness-app
3 JWT_SECRET=remplace_par_une_cle_secrete_longue_et_aleatoire
```

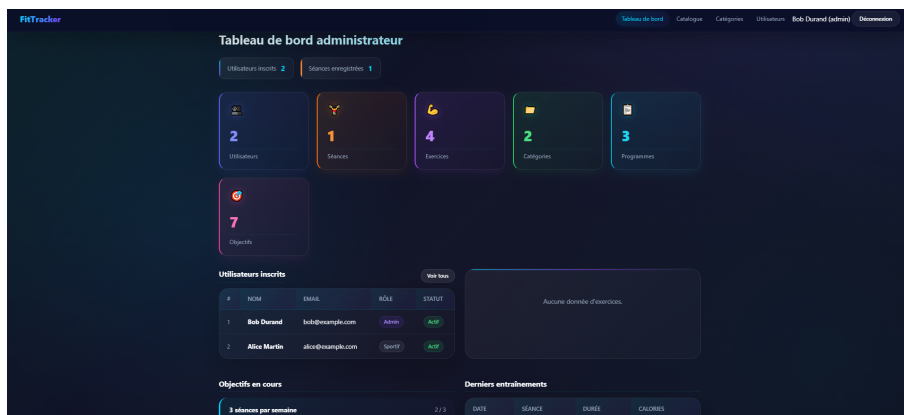



FIGURE 7 – Tableau de bord admin

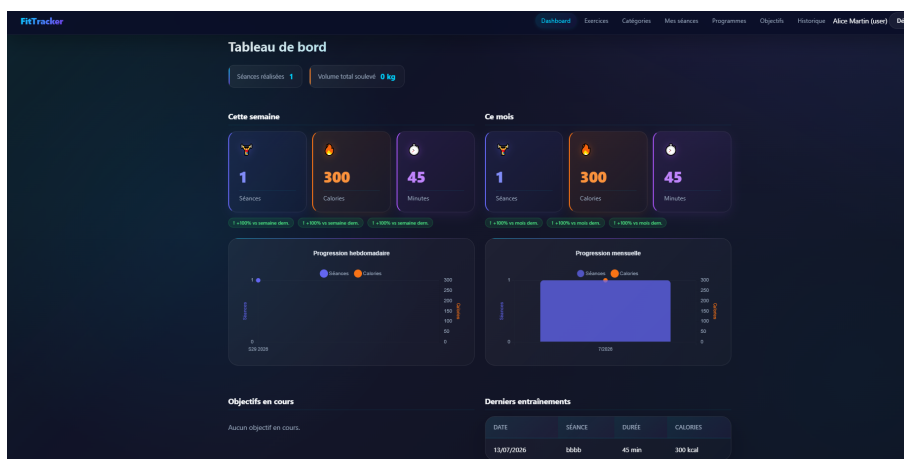


FIGURE 8 – Tableau de bord d'un utilisateur avec statistiques hebdomadaires

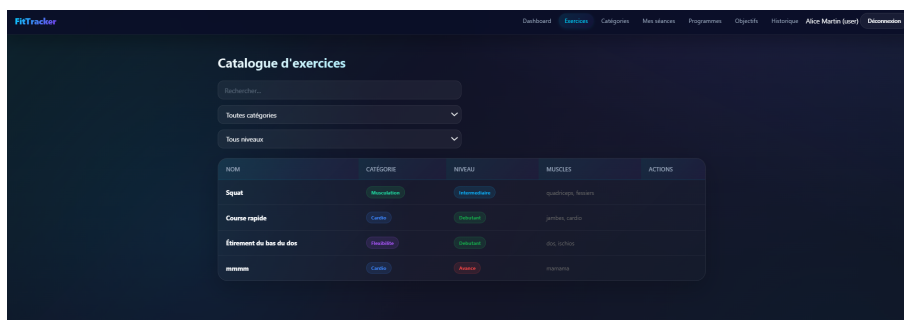
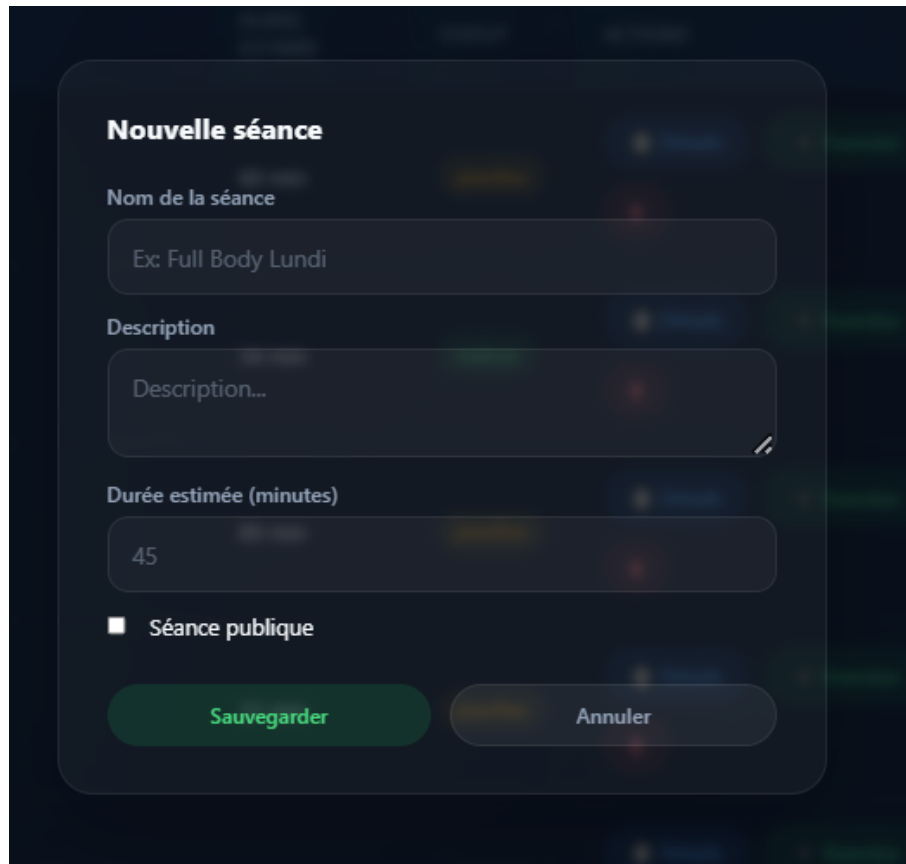


FIGURE 9 – Catalogue d'exercices



Nouvelle séance

Nom de la séance

Ex: Full Body Lundi

Description

Description...

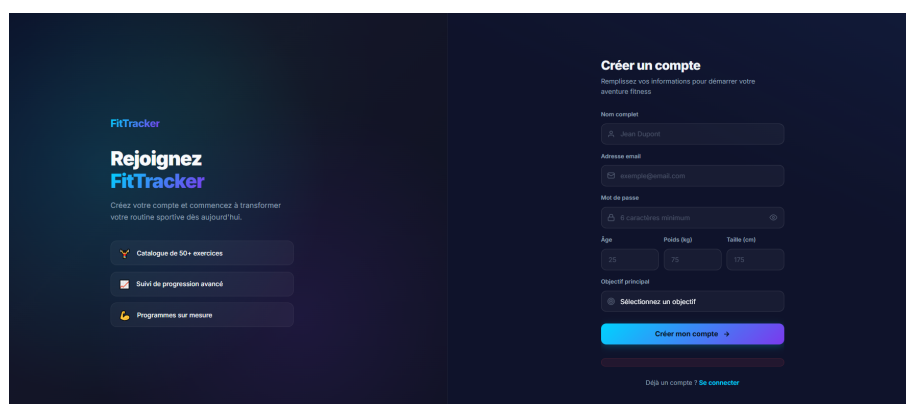
Durée estimée (minutes)

45

☒ Séance publique

Sauvegarder **Annuler**

FIGURE 10 – Formulaire de création d'une séance



FitTracker

Rejoignez FitTracker

Créez votre compte et commencez à transformer votre routine sportive dès aujourd'hui.

- Catalogue de 50+ exercices
- Suivi de progression avancé
- Programmes sur mesure

Créer un compte

Remplissez vos informations pour démarrer votre aventure fitness

Nom complet

Adresse email

Mot de passe

8 caractères minimums

Âge

Palais (kg)

Taille (cm)

Objectif principal

Sélectionnez un objectif

Créer mon compte →

Déjà un compte ? [Se connecter](#)

FIGURE 11 – Page d'inscription

```
4 JWT_EXPIRES_IN=24h
```

3. Initialisation des données de test Depuis le dossier `backend`, lancer le script d'insertion des données de test :

```
1 node seedData.js
```

4. Démarrage du serveur

```
1 npm run dev
2 # Le serveur démarre sur http://localhost:5000
```

5. Lancement du frontend Ouvrir le fichier `frontend/index.html` dans un navigateur, ou servir le dossier `frontend/` avec un serveur statique (par exemple l'extension « Live Server » de VS Code) pour éviter les restrictions liées au protocole `file://`.

6. Connexion de test

Rôle	Email	Mot de passe
Administrateur	bob@example.com	Password123
Utilisateur	alice@example.com	Password123

Annexe D — Répartition des tâches

Nom et prénom	Tâches réalisées	% de contribution
INEZA Benisse	Backend : authentification, modèles Mongoose, middleware, rédaction du rapport	35 %
GAHIMBARE Benitha	Backend : contrôleurs des routes métier (séances, programmes, objectifs)	25 %
EZAKO Juste Ariel	Frontend : intégration des pages HTML/CSS/JS et appels API	20 %
IKORIVYZA Seth	Modélisation NoSQL, tests	20 %